Intelligent Wi-Fi Relay Drone for Autonomous Disaster Communication.

Final Project Report — Machine Vision

Vision-guided landing-spot detection for a self-deploying relay-drone chain, developed and evaluated in simulation

TEAM MEMBERS:

LOKESHWARAN P – 23BAI1098

ANNAMALAI SRINIVAS – 23BAI1006

YATHARTH TIWARI – 23BAI1401

Code, trained models and results: github.com/cosmosapien200/communication-drone-relay-sim

Table of Contents

1. Abstract **3**

2. Introduction **3**

3. Problem Statement **4**

4. Literature Review **5**

4.1 Drone-based emergency communication (from Review 1) 5

4.2 Semantic segmentation 6

4.3 Synthetic data and domain randomisation 6

4.4 Vision-based safe landing for UAVs 6

4.5 Mapping, exploration and learning-based control 6

4.6 Research gap 7

5. Dataset Description **7**

5.1 Generation pipeline 7

5.2 Classes 8

5.3 Splits and statistics 8

6. Methodology **9**

6.1 System overview 9

6.2 Landable-surface segmentation network 9

6.3 Ground projection 10

6.4 Evidence fusion into the shared landable map 10

6.5 Camera gimbal control 11

6.6 Landing-spot locator 11

6.7 Vision noise model 11

6.8 Reinforcement learning, flight and communication (summary) 11

6.9 Overall vision pipeline 12

7. Implementation **12**

7.1 Code organisation 13

7.2 Training procedure 13

7.3 Deployment inside the simulator and verification 13

7.4 Simulation demonstration 14

8. Experimental Results **14**

8.1 Segmentation accuracy 15

8.2 Accuracy by viewing condition 16

8.3 Qualitative results and failure cases 17

8.4 Landing-spot localisation 18

8.5 Landing with vision in the loop 19

8.6 Mission-level effect of perception 19

9. Performance Evaluation **20**

9.1 Comparison with Review 1 objectives 20

9.2 Strengths 21

9.3 Limitations 21

9.4 Failure analysis of the live demonstration 21

10. Conclusion **22**

11. Future Scope **23**

12. References **23**

1. Abstract

Natural disasters destroy communication infrastructure exactly when rescue teams need it most. This project builds an intelligent Wi-Fi relay drone system in which drones deploy one after another from a base station and land on camera-detected spots, forming a relay chain that carries the network toward a disconnected disaster camp. A landed relay draws only radio power and can stay on station far longer than a hovering one, which makes reliable landing-spot perception the central machine-vision problem of the system.

Because of the limited project time, the complete system was developed and validated in simulation: a procedurally generated collapsed town in Unity 6 with a radio propagation model, a simulated quadrotor flight controller and two reinforcement-learning policies (navigation and landing). The machine-vision stack consists of (i) a fully synthetic, automatically labelled dataset of 13,750 images rendered from 125 generated towns with domain randomisation, plus 9,120 targeted fine-tuning images; (ii) a lightweight U-Net (0.48 M parameters, 0.59 GMAC) that segments every 128×128 camera frame into landable, not-landable, obstacle and unknown classes; (iii) a geometric ground-projection and evidence-fusion module that turns per-pixel predictions into a shared landable-surface map; and (iv) a landing-spot locator that converts the nadir segmentation into observations for the landing policy.

On held-out test towns the final model reaches a mean IoU of 0.919 and a landable-class IoU of 0.825 (precision 0.89, recall 0.92), runs inside the simulator in about 4 ms per frame on a CPU, and locates the target patch in 98.8 % of nadir frames with a median centre error of 0.44 m. In closed-loop missions the detector's quality directly controls how far the relay chain reaches: the chain is robust up to roughly twice the measured detector error and collapses beyond four times it. Failure analysis of the live demonstrations traced most failures to perception and led to three vision improvements (goal-aimed camera gimbal, geometry-weighted evidence fusion, and fine-tuning on mission-specific viewpoints); together with replacing two procedural towns that proved unreliable, demonstration-map reliability rose from 34/60 to 58/60 runs.

2. Introduction

Floods, earthquakes, cyclones and landslides routinely damage cellular towers, power and fibre, producing communication blackouts. Rescue teams lose contact with command centres, victims cannot share their location, and coordination slows down. Drones are an attractive way to restore connectivity quickly, but a single hovering drone

is limited by battery life (minutes), and in a dense, collapsed town radio signals are blocked by buildings, so one relay is rarely enough.

The approach taken here is a chain of relays that perch: each drone flies out from the end of the existing chain, explores the streets toward the disaster camp while keeping a usable link to its parent, finds a safe landing surface and lands there as a relay. The next drone inherits a shared map and continues from that relay. Whether this works depends critically on the drone's ability to see: it must recognise which surfaces are safe to land on (intact concrete, plazas, intact road) and which only look similar (debris, puddles, holes, wrecked vehicles, slopes), localise the chosen surface in metric world coordinates, and track it precisely during descent under wind gusts.

This report focuses on these machine-vision components. Review 1 proposed a camera-plus-reinforcement-learning relay drone; the final work keeps that idea but moves the entire development into simulation, which allowed us to generate unlimited labelled training data, evaluate on hundreds of held-out scenarios, and run the full multi-drone mission end to end within the project time. Simulation also let us measure, for the first time in this project, how vision quality translates into communication range.

- **Contribution 1 – Synthetic perception dataset:** a fully automatic pipeline that renders photorealistically randomised town images with pixel-perfect labels by re-rendering the same scene with flat class colours.
- **Contribution 2 – Landable-surface segmentation:** a 0.48 M-parameter U-Net, trained in PyTorch, exported to ONNX and executed inside the simulator in real time.
- **Contribution 3 – From pixels to a map:** ground projection with the known camera pose and a geometry-weighted evidence fusion that builds a shared landable-surface map across drones.
- **Contribution 4 – Landing-spot localisation:** connected-component and distance-transform analysis of the nadir segmentation, calibrated against ground truth to build a measured noise model for reinforcement learning.
- **Contribution 5 – Closed-loop evaluation:** measuring how segmentation quality changes the range of the relay chain, and a failure analysis that turned live-demo failures into concrete vision improvements.

3. Problem Statement

Given a disaster area where the base station cannot reach the target camp directly, deploy a sequence of drones that autonomously extend network coverage toward the camp by landing at suitable spots, such that every link in the chain stays above a minimum signal strength (−80 dBm) and the camp is finally covered.

The machine-vision sub-problem is:

- From a single 128×128 RGB camera on a gimbal (45° forward-down while exploring, straight down while landing), classify every pixel as landable, not landable, obstacle or unknown, in a cluttered environment full of non-landable look-alikes and under varying light, shadow and dust.
- Project these predictions onto the ground using the drone's known pose and accumulate them over time and across drones into a reliable landable-surface map, so that the mission planner can choose a landing spot that both advances the chain and has enough signal.
- During landing, localise the chosen patch in the nadir image (offset, apparent size, confidence and edge clearance) robustly enough that a learned landing controller can touch down inside it under wind gusts.
- Do all of this with a model small enough for embedded deployment (the Review 1 target platform was an RV1103-class processor), and quantify how detector errors propagate to mission-level performance.

Constraints: no real flight data was available in the project time; the solution therefore had to be trained and validated with synthetic data and simulation, with careful separation of training and evaluation maps so that results reflect generalisation to unseen towns.

4. Literature Review

4.1 Drone-based emergency communication (from Review 1)

FSO-based drone-assisted mobile access networks use drones as temporary aerial base stations connected to surviving infrastructure through free-space optical links and optimise placement, user association and bandwidth. They deploy quickly but depend on surviving infrastructure [1].

Multi-UAV wireless mesh networks form a self-organising Wi-Fi mesh for disaster recovery with real-time video and target tracking, demonstrating low latency in field tests, but rely on 5G and non-terrestrial network infrastructure, which raises cost and complexity [2].

Blockchain-enhanced UAV networks secure multi-agency coordination with smart contracts and hybrid consensus, gaining resilience at the price of significant computational and energy overhead [3].

Across these works, drone positions are usually predetermined or optimised centrally; none addresses how a drone perceives where it can physically perch to act as a long-endurance relay.

4.2 Semantic segmentation

Fully convolutional networks [4] made dense per-pixel classification practical by replacing fully connected layers with convolutions. U-Net [5] added a symmetric decoder with skip connections that recover fine spatial detail, and remains a strong choice when training data is limited and boundaries matter. Dice-based losses [6] help with class imbalance, which is relevant here because landable surfaces cover only 6–17 % of pixels. Embedded-oriented models such as FastDepth [7], which Review 1 considered for depth estimation, show that encoder–decoder networks well under one million parameters can run on low-power processors.

4.3 Synthetic data and domain randomisation

Rendering engines can produce pixel-perfect labels at no annotation cost. Richter et al. [8] extracted ground truth from a video game and showed that synthetic data reduces the amount of real data needed; SYNTHIA [9] built a virtual city specifically for urban segmentation. Tobin et al. [10] showed that randomising textures, lighting and camera parameters broadly enough makes models treat the real world as just another variation. Our dataset follows these ideas: every image is generated from a procedural town with randomised palette, sun direction, dust and fog, and labels come from a second, flat-colour render of the same view.

4.4 Vision-based safe landing for UAVs

Safe-landing-zone detection has been studied with classical texture and slope analysis [11] and with learned semantic segmentation for urban search and rescue, where landing candidates are scored from segmented aerial images [12]. These works typically assume a single nadir view; our setting additionally needs oblique exploration views to find candidates from street level and a nadir tracker during descent.

4.5 Mapping, exploration and learning-based control

Occupancy grids [13] fuse uncertain observations into a probabilistic map; our landable-surface map is a per-cell evidence grid in the same spirit. Frontier-based exploration [14] drives exploration toward the boundary between known and unknown space and is used by our planner. The flight policies are trained with Proximal Policy Optimization [15] in Unity ML-Agents [16]; the radio channel follows the log-distance path-loss model [17] with material wall losses in the range reported by ITU-R P.2040 [18].

4.6 Research gap

Existing relay-drone work focuses on network optimisation and assumes the drones can hold position; landing-zone detection work focuses on a single landing. There is little work that (a) uses vision to let a sequence of relay drones perch on safe surfaces, (b) fuses detections across drones into a shared map, and (c) measures how perception quality changes the communication range achieved. This project addresses that gap in simulation.

5. Dataset Description

Review 1 listed KITTI and NYU Depth V2 for depth estimation. Neither provides labels for the question this system must answer ("can a drone land here?"), neither covers collapsed urban scenes from a drone's viewpoint, and there was no time to collect and annotate real flight imagery. We therefore generated a synthetic dataset directly from the simulator, with labels defined exactly by the task.

5.1 Generation pipeline

- **Scene:** a seeded procedural collapsed town (96–120 m square, 1 m grid): street grid, box buildings 8–20 m tall, collapsed buildings and rubble barriers, landable surfaces (plaza tiles, flat concrete, intact road and street-aligned intact-road strips) and five kinds of look-alikes (debris fields, puddles, holes, wrecked vehicles, ramps/slopes).
- **Rendering:** Unity built-in pipeline with real-time shadows; ground painted procedurally at 16 px/m with cracks, patches and dust; buildings and rubble from box primitives with procedural facade textures.
- **Domain randomisation (per map):** asphalt tone, dust tint and amount, surface palettes, sun elevation 15–75° and azimuth 0–360°, sun intensity and colour, ambient level, sky colour, fog in 30 % of maps.
- **Camera poses:** 45° forward-down at cruise altitude 5 m (exploration) and nadir at 0.6 –6 m (landing), 60° field of view, 128×128 pixels; nadir views are biased toward ground features.
- **Automatic labels:** the same frame is rendered a second time with every material replaced by an unlit flat class colour and no lighting or fog; the colours are decoded to class indices, so labels are pixel-perfect and need no manual annotation.

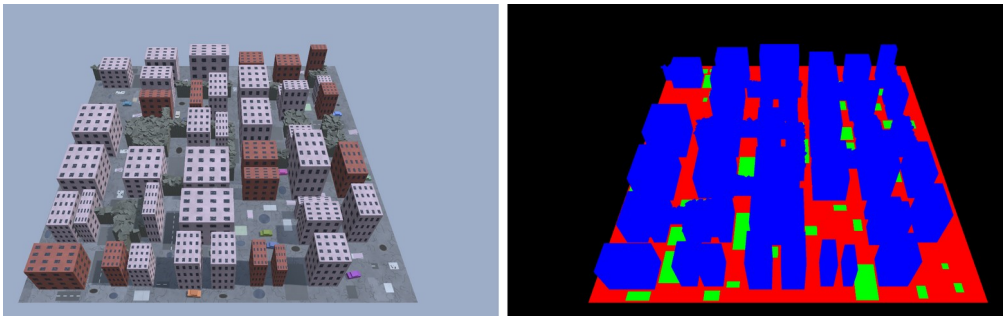


Figure 1. Label generation by re-rendering: the same generated town rendered normally (left) and with flat class colours (right). Green = landable, red = not landable, blue = obstacle, black = unknown/sky.

5.2 Classes

Table 1. Segmentation classes

Class	Definition	Typical content
Landable	Flat, intact surface large enough for the drone	Plaza tiles, flat concrete, intact road and road strips
Not landable	Ground that is unsafe	Cracked asphalt, debris, puddles, holes, wrecked vehicles, ramps
Obstacle	Anything above the ground plane	Buildings, rubble piles
Unknown	No ground information	Sky, outside the map

5.3 Splits and statistics

Images are split by map seed, so every test image comes from a town never seen during training (training seeds are even, validation and test seeds odd and in separate ranges). After the first mission experiments two targeted fine-tuning sets were added, again from new maps: close-range nadir frames (0.5–2.5 m) and 45° views aimed at a landable patch or look-alike from 5–20 m, which is what the exploration camera actually sees during a mission.

Table 2. Dataset splits and pixel class balance of the training split (all splits use disjoint map seeds)

Set	Maps (train / val / test)	Frames (train / val / test)	Landable	Not landable	Obstacle	Unknown
Main (45° + nadir)	100 / 10 / 15	11,000 / 1,100 / 1,650	16.6 %	61.2 %	16.2 %	6.0 %
Close-range nadir	60 / – / 12	3,600 / – / 720	24.7 %	74.3 %	0.8 %	0.2 %
Patch-aimed 45°	80 / – / 15	4,769 / – / 884	14.4 %	67.0 %	14.6 %	4.0 %

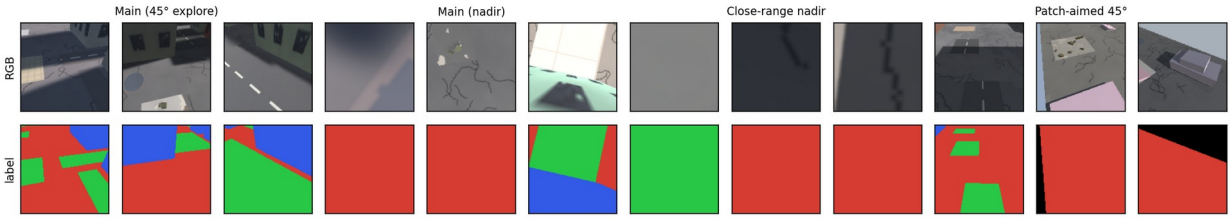


Figure 2. Dataset samples (top: rendered input, bottom: automatic label) for the four view types.

The class distribution is imbalanced (landable 14–25 % of pixels, not-landable 61–74 %), which is handled with class-weighted cross-entropy plus Dice loss (Section 6.2).

6. Methodology

6.1 System overview

Due to the limited time available for hardware integration and field testing, the complete system was designed, trained and validated in simulation. The simulation reproduces the elements that matter for perception and decision making: camera images, range sensors, radio signal strength, wind and quadrotor dynamics. The figure below shows the architecture. A scripted mission planner decides where each drone explores and lands; two PPO policies fly; the machine-vision module feeds both the planner (landable map) and the landing policy (spot observations).

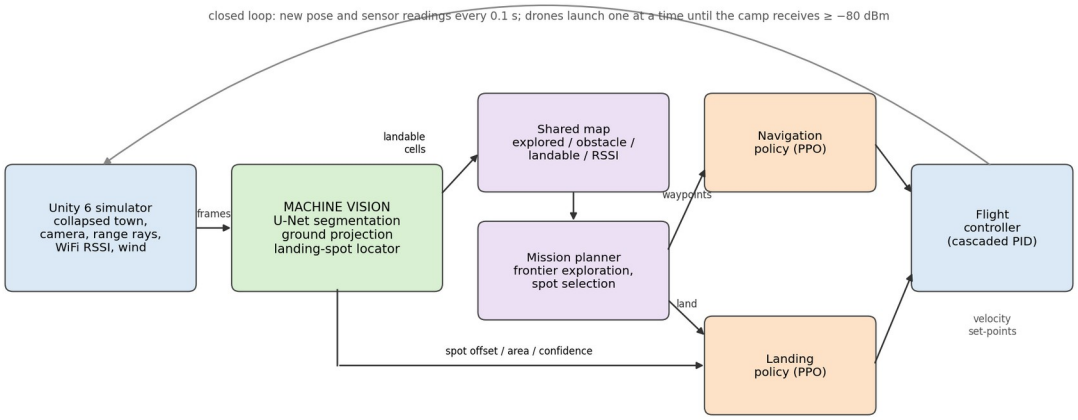


Figure 3. System architecture. The machine-vision module (green) converts camera frames into landable cells for the shared map and into spot observations for the landing policy.

6.2 Landable-surface segmentation network

A compact U-Net [5] was chosen over heavier segmentation models because the target platform is embedded and the input resolution is small. The encoder has four levels (16, 32, 64, 128 channels), each with two 3×3 convolution–batch-norm–ReLU blocks and 2×2

max pooling; the decoder mirrors it with transposed convolutions and skip connections, and a 1×1 convolution outputs four class logits. The exported model applies a softmax and returns per-pixel class probabilities; the maximum probability is used as confidence.

Table 3. Segmentation model configuration

Property	Value
Input / output	RGB 3×128×128 → class probabilities 4×128×128
Parameters / compute	0.48 M / 0.59 GMAC per frame
Loss	class-weighted cross-entropy (weights ∝ 1/√frequency) + 0.5 × soft Dice
Optimiser / schedule	AdamW (lr 3e-3, weight decay 1e-4), OneCycle, batch 64, 40 epochs from scratch
Augmentation (on GPU)	brightness, contrast, saturation and tint jitter, random blur, Gaussian noise, horizontal flip
Fine-tuning (v3 → v4)	initialised from previous model, lr 1e-3, 15 epochs, original + targeted sets
Deployment	ONNX (opset 15, 1.9 MB) → Unity Inference Engine

6.3 Ground projection

Each predicted pixel (u, v) is turned into a ray using the pinhole model of the 60° camera: with t = tan(30°), the ray direction in camera coordinates is d = (t(2(u+0.5)/W − 1), t(1 − 2(v+0.5)/H), 1), rotated by the known gimbal orientation. The ray is intersected with the ground plane y = 0 at s = −y_cam / d_y. Pixels whose ray points above the horizon, whose ground hit is farther than 25 m, or whose line of sight is blocked by a known obstacle column are discarded; obstacle and unknown pixels are not projected. The remaining hits vote for the 1 m map cell they fall in, using every second pixel (4,096 rays per frame).

6.4 Evidence fusion into the shared landable map

Each cell keeps two accumulators, landable evidence E_L and not-landable evidence E_N. A projected pixel adds a weight w = p_max · max(0, −d_y) · (5 m / max(5 m, r))² to the accumulator of its predicted class, where p_max is the network confidence, −d_y the cosine of the viewing angle from vertical (steep views are more reliable) and r the ground distance. A cell is landable when E_L ≥ 0.5 and E_L / (E_L + E_N) ≥ 0.6, and a landing spot must additionally have at least three of its four neighbours landable (drone footprint plus clearance). The inverse-square range term was introduced after failure analysis showed that many distant, blurred, grazing-angle observations were outvoting a few close, correct ones (Section 9.4). Each drone works on a copy of the shared map and uploads only the evidence it gathered itself after landing.

6.5 Camera gimbal control

While exploring, the 45° camera is aimed at the planner's current goal (frontier cell or candidate spot), falling back to the direction of travel, with a ±60° sinusoidal pan (4 s period) to cover side streets. During landing the gimbal points straight down. Aiming the camera independently of the body was a key finding: the navigation policy controls yaw freely and often flies sideways, so a body-fixed camera frequently never looked at the spot the drone was flying to.

6.6 Landing-spot locator

In the nadir view the locator thresholds the landable probability at 0.5, labels 4-connected components, and selects the component of at least 12 pixels closest to where the target patch is expected from the drone's pose (within 35 % of the image width). It outputs the component centroid offset from the image centre (normalised to [−1, 1] in the drone's right/forward axes), the apparent area fraction, the mean landable probability (confidence) and the edge clearance, i.e. the Chebyshev distance from the centroid to the nearest non-component pixel converted to metres with the ground sample distance $2 \cdot h \cdot \tan(30°)/128$. These ten-dimensional observations (with altitude, velocity and frames since last detection) are the landing policy's input.

6.7 Vision noise model

Rendering cameras is the expensive part of simulation, so the reinforcement-learning policies were trained against a noise model with the same output types as the real modules. The noise model is calibrated from the real detector on held-out maps: the row-normalised confusion matrix corrupts ground-truth cell labels, and the measured locator false-negative rate, false-positive rate, localisation standard deviation and bias, area error and dropout corrupt ground-truth spot detections. Switching between the noise model and the real network is a single configuration option, which also enables a controlled detector-quality sweep (Section 8.6).

6.8 Reinforcement learning, flight and communication (summary)

- **Navigation policy:** PPO, MLP 2×256, 33 inputs (waypoint distance and bearing, body velocity, heading, 24-ray range ring up to 10 m, previous action), 3 velocity outputs; reward = normalised shortest-path progress − time cost − proximity penalty, −1 on collision, +1 at the waypoint; curriculum over four town difficulty levels.
- **Landing policy:** PPO, MLP 2×128, the 10 vision-based inputs of Section 6.6; reward penalises offset and fast descent near the ground with potential-based shaping, +1 plus a centring bonus for a soft touchdown inside the patch, −1 for off-patch, hard impact or losing the patch; gust curriculum 0 → 4.5 m/s; fine-tuned with the measured vision noise.

- **Flight controller:** velocity set-points to a simulated quadrotor (cascaded velocity → attitude → rate PID, four rotors); a fast first-order model with matched step response is used for training.
- **Radio:** log-distance path loss (−40 dBm at 1 m, exponent 2.0) minus 10/7/5 dB per concrete/brick/rubble wall crossed; a relay spot qualifies if the measured signal to its parent is at least −71 dBm (link threshold −80 dBm + 6 dB margin + 3 dB ground penalty).

6.9 Overall vision pipeline

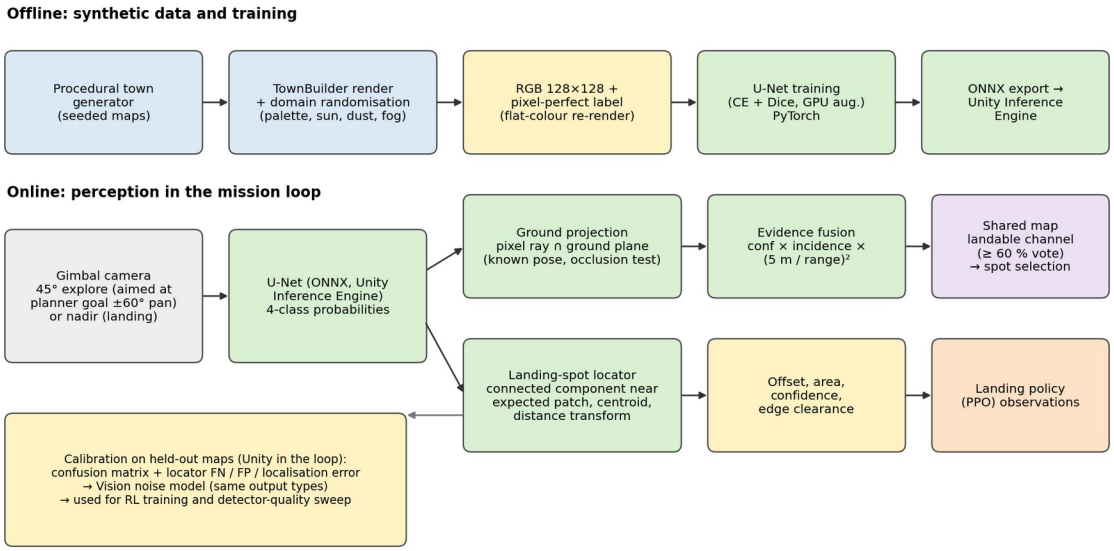


Figure 4. Machine-vision pipeline: offline synthetic data generation and training (top), online perception in the mission loop (bottom), and calibration of the noise model.

7. Implementation

Table 4. Implementation stack

Component	Technology
Simulator	Unity 6 (6000.6.4f1), built-in render pipeline, C#; all scenes and maps built from code
Segmentation training	PyTorch 2.5 (CUDA 12.1) on an NVIDIA RTX 3060 12 GB; data and augmentation kept on the GPU
In-simulator inference	ONNX → Unity Inference Engine 2.6 (CPU backend), 128×128 render target
Reinforcement learning	Unity ML-Agents 4.1 + mlagents 1.1.0 (PPO), CPU, 32 headless arenas per process
Tooling	uv project environment (Python 3.10), Unity MCP for editor inspection, git
Compute	Intel i5-13400F (16 threads), 32 GB RAM, RTX 3060

Component	Technology
	12 GB

7.1 Code organisation

The vision code is split along the interfaces of the system: the dataset generator (VisionDataset.cs) and scene renderer with label mode (TownBuilder.cs) in Unity; training, evaluation and ONNX export in PyTorch (train_seg.py); and in the simulator a model wrapper (SegModel.cs), the segmenter with ground projection and weighted observations (VisionSegmenter.cs), the landing-spot locator (LandingSpotLocator.cs), the calibrated noise model (VisionCalibration.cs) and the shared map with evidence accumulation (SharedMap.cs). All scenes and maps are built from code, and every experiment is reproducible from a seed.

7.2 Training procedure

The main model (v2) was trained from scratch for 40 epochs on 11,000 images (about 20 s per epoch on the GPU). The deployed model (v4) was obtained by two fine-tuning steps: v3 added 3,600 close-range nadir images, and v4 added 4,800 patch-aimed exploration images, each for 15 epochs at a lower learning rate while still training on the original data to avoid forgetting. Two deliberately weaker models trained on 5 % and 20 % of the training maps serve the detector-quality ablation.

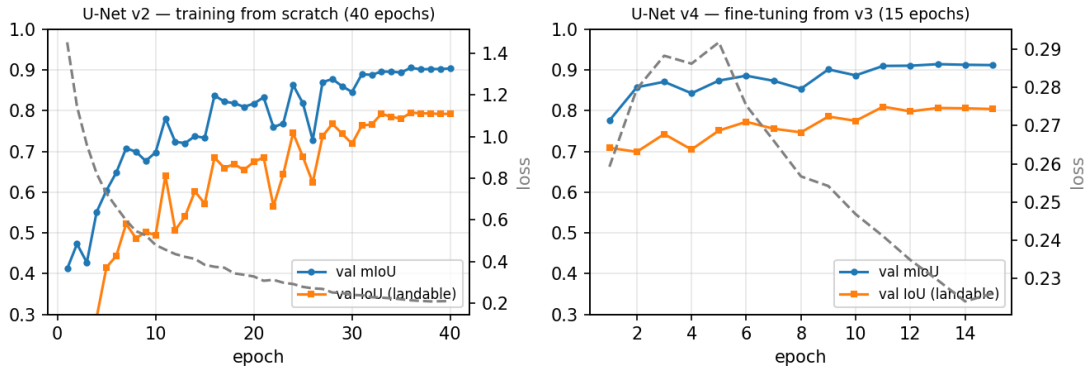


Figure 5. Validation mIoU, landable IoU and training loss: training from scratch (left) and the final fine-tuning step (right).

7.3 Deployment inside the simulator and verification

The ONNX model runs in Unity's Inference Engine on rendered 128×128 frames. Two integration issues were found and fixed through explicit verification rather than assumed: (1) image orientation — the network was run inside Unity on test images and compared with the labels (94.5 % pixel accuracy with the correct top-left convention versus 62.6 % if flipped); (2) the GPU compute backend returned constant outputs in the

Linux editor, so the CPU backend is used (≈4 ms per frame). A third, subtler issue was non-determinism between the windowed demo and headless evaluation: Unity sizes the shadow map from the screen resolution, so shadows and thus predictions differed slightly; a fixed shadow resolution made both identical.

Measured inference cost of the deployed network (0.48 M parameters, 0.59 GMAC, 1.9 MB ONNX): 12.4 ms per frame on one CPU thread, 4.1 ms on four threads and 0.68 ms on the RTX 3060 GPU — comfortably within the 10 Hz decision rate and small enough for an embedded NPU.

7.4 Simulation demonstration

A live demonstration runs in the Unity editor or as a standalone build with six maps (four hand-built layouts and two held-out procedural towns). The on-screen display shows the mission mode, signal to parent, weakest link, battery, detection confidence and reward, a minimap of the shared landable map, and the camera view with the live segmentation overlay.

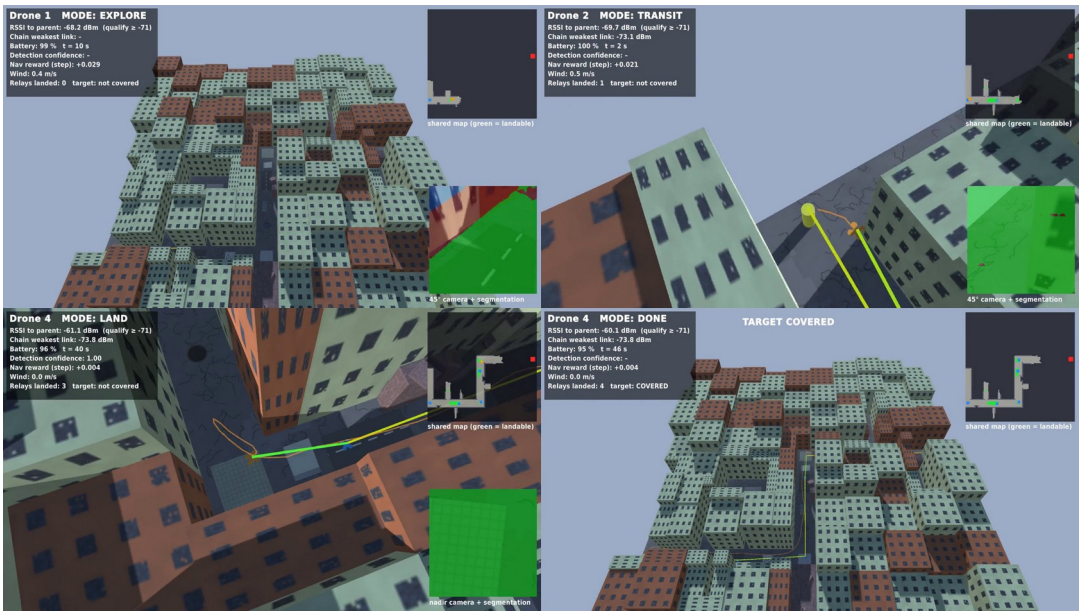


Figure 6. Frames from the demonstration video: exploration with the 45° camera and live segmentation (top left), navigating to a detected spot (top right), nadir landing with detection confidence 1.00 (bottom left), and the completed four-relay chain with the camp covered (bottom right).

8. Experimental Results

All results are on held-out map seeds never used for training. Segmentation metrics are pixel-level over the 1,650-image main test set unless stated; mission results average 100 missions per condition.

8.1 Segmentation accuracy

Table 5. Per-class results of the final model (U-Net v4) on held-out test towns

Class	IoU	Precision	Recall
Unknown	0.962	0.967	0.995
Landable	0.825	0.888	0.921
Not landable	0.934	0.972	0.960
Obstacle	0.952	0.978	0.973
Mean / pixel accuracy	**0.919**		**95.7 %**

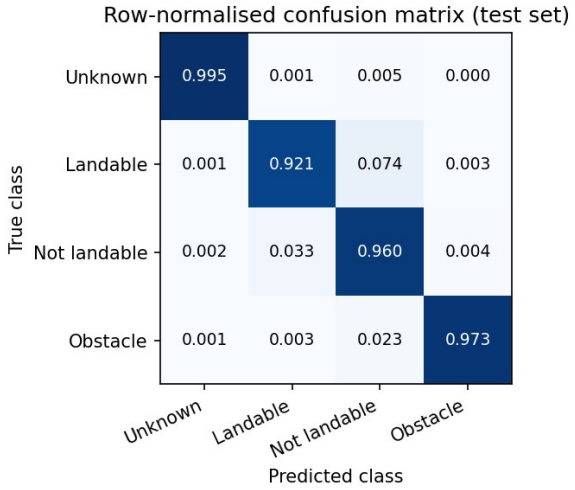


Figure 7. Row-normalised confusion matrix of the final model. Most residual landable errors are confusions with not-landable ground.

Table 6. Model comparison on the same test set

Model	Training data	mIoU	IoU landable	Precision landable	Recall landable
Weak (5 % data)	5 maps	0.476	0.190	0.361	0.287
Mid (20 % data)	20 maps	0.656	0.422	0.619	0.569
U-Net v1	100 maps (early world)	0.848	0.636	0.929	0.669
U-Net v2	100 maps	0.909	0.810	0.880	0.910
U-Net v4 (final)	100 maps + 132 fine-tune maps	0.919	0.825	0.888	0.921

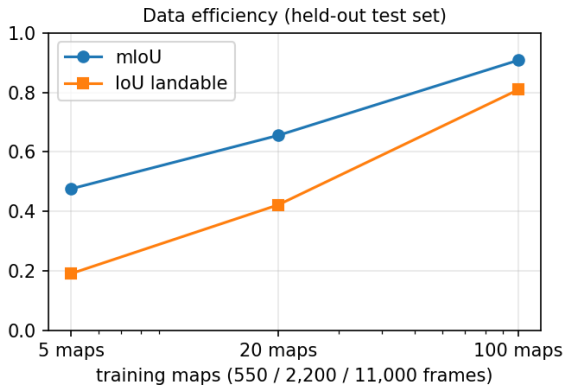


Figure 8. Data efficiency: segmentation quality as a function of the number of training towns.

8.2 Accuracy by viewing condition

Table 7. Landable-class accuracy by camera view (v2 = before fine-tuning, v4 = final)

Condition	v2 IoU landable	v4 IoU landable	v4 precision	v4 recall	v4 mIoU
45° explore view	0.861	**0.880**	0.937	0.935	0.950
Nadir view (all)	0.779	**0.793**	0.859	0.912	0.822
Nadir, altitude < 2 m	0.565	**0.595**	0.653	0.871	0.479
Nadir, altitude > 4 m	0.892	**0.891**	0.960	0.925	0.907
Close-range nadir set (0.5–2.5 m)	0.578	**0.617**	0.654	0.917	0.525
Patch-aimed 45° set (5–20 m)	0.787	**0.874**	0.936	0.930	0.939

The network is strongest on oblique exploration views (landable IoU 0.88) and on nadir views from 4–6 m (0.89). Close-range nadir views below 2 m remain the hardest case (0.60–0.62): at that height the camera sees only a few ground-texture texels, shadows dominate, and a uniformly grey frame is genuinely ambiguous. Recall stays high (0.87–0.92) there, so the landing locator rarely loses the patch, but precision drops.

8.3 Qualitative results and failure cases

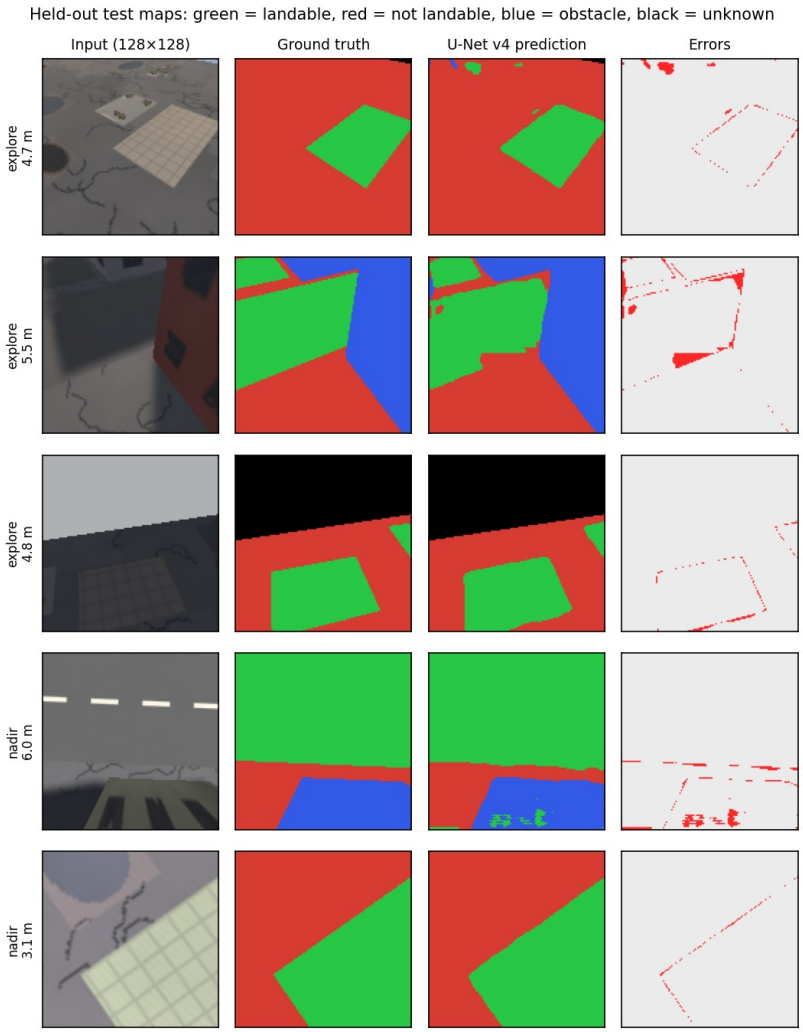


Figure 9. Predictions on held-out test towns. Errors (right) concentrate on thin bands along class boundaries.

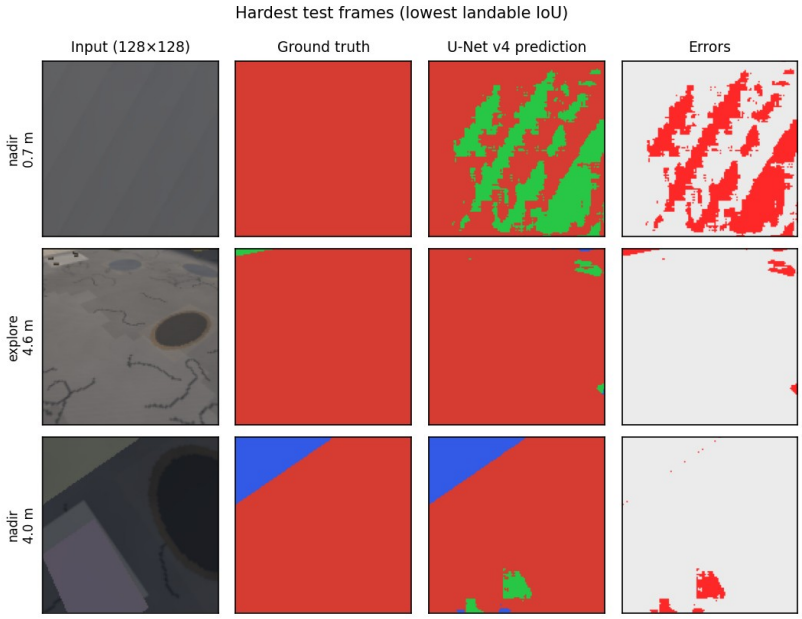


Figure 10. The hardest test frames: a featureless close-range nadir view (top), a distant small patch (middle) and a shaded patch next to a hole (bottom).

8.4 Landing-spot localisation

Table 8. Landing-spot locator measured inside the simulator on 864 nadir frames from 15 held-out towns

Detector	False negatives	False positives (patch out of view)	Median centre error	90th percentile
U-Net v1	7.3 %	41.7 %	0.33 m	0.84 m
U-Net v2	1.6 %	40.2 %	0.44 m	1.12 m
U-Net v4 (final)	**1.2 %**	44.6 %	**0.44 m**	1.15 m
Mid (20 % data)	19.4 %	16.1 %	0.57 m	1.42 m
Weak (5 % data)	100 %	0 %	–	–

False negatives are rare for the trained models. The high false-positive rate when the target is out of view means the locator usually reports some other landable surface nearby; the landing policy copes with this because it was trained with exactly this measured noise, and the planner only lands after the spot has been confirmed from cruise altitude. The weak model never isolates the patch, which makes it a useful lower bound.

8.5 Landing with vision in the loop

Table 9. Landing success and touchdown error with the measured detector noise (100 episodes per cell)

Gust (m/s)	RL landing policy	PID visual servoing
0	100 %, error 0.01 m	97 %, error 0.02 m
1.5	100 %, 0.02 m	91 %, 0.04 m
3.0	100 %, 0.02 m	76 %, 0.07 m
4.5	100 %, 0.03 m	64 %, 0.11 m
6.0 (beyond training)	100 %, 0.05 m	44 %, 0.14 m
3.0 with 2× detector noise	100 %	0 %

Training the landing controller on the calibrated detector noise is what makes it robust: the classical controller chases false detections and loses the patch, while the learned policy holds its approach when the detection is unreliable and lands within a few centimetres.

8.6 Mission-level effect of perception

Table 10. Full missions on 100 held-out level-2 towns (up to six drones)

Perception used by the drones	Camp covered	Range extended toward camp (m)
Ground truth (perfect vision)	43 %	63.6 ± 34.0
Real U-Net v2 in the loop	34 %	58.1 ± 36.2
Mid detector in the loop	7 %	27.9 ± 29.7
Weak detector in the loop	0 %	0.0
Ground truth, final system	51 %	70.3 ± 34.0
Real U-Net v4 in the loop, final system	**45 %**	**62.3 ± 35.0**

With the final vision system (goal-aimed gimbal, geometry-weighted fusion and U-Net v4) the share of missions that cover the camp with the real detector in the loop rose from 34 % to 45 %, and the range extended from 58.1 m to 62.3 m. The same planner with perfect vision reaches 51 % (70.3 m), so the remaining gap attributable to perception is about 6 percentage points; the rest is limited by town geometry and radio range.

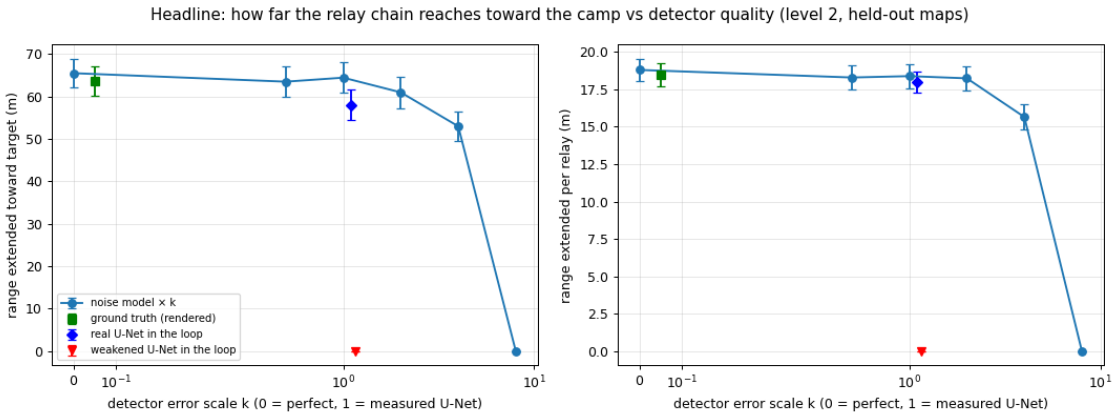


Figure 11. How far the relay chain reaches toward the camp as the detector's error rates are scaled by k (k = 0 perfect, k = 1 measured U-Net, k > 1 weakened), with the real networks in the loop as separate markers.

Table 11. Detector-quality sweep with the calibrated noise model (100 missions per value)

Error scale k	0	0.5	1	2	4	8
Camp covered	45 %	51 %	47 %	45 %	30 %	0 %
Range extended (m)	65.5	63.5	64.4	61.0	53.0	0.0

This is the central result connecting vision to communication: mission performance is flat up to about twice the measured error of our detector, degrades beyond four times, and collapses when the detector can no longer isolate landable surfaces. The real network in the loop performs slightly worse than its noise-model twin, because real errors are spatially correlated (a whole patch is misread) whereas the noise model assumes independent pixels.

9. Performance Evaluation

9.1 Comparison with Review 1 objectives

Table 12. Objectives versus outcomes

Objective (Review 1)	Outcome
Lightweight perception suitable for an embedded processor	U-Net with 0.48 M parameters, 0.59 GMAC, 1.9 MB; 4 ms per frame on 4 CPU threads
Autonomous relay position selection	Vision-built landable map + RSSI-gated spot selection; relays land 18 m further along the path per relay on average

Objective (Review 1)	Outcome
Improved coverage / signal strength	With the real detector the chain extends coverage 62 m toward the camp on average and covers it in 45 % of held-out level-2 missions; weakest link ≈ −72.5 dBm (above the −80 dBm threshold)
PPO-based decision making	Navigation 98/98/93/80 % success on four difficulty levels; landing 100 % under gusts up to 6 m/s with real detector noise
Simulation before deployment	Entire pipeline validated in simulation; hardware deployment is future work

9.2 Strengths

- Zero manual annotation: unlimited, pixel-perfect labels and held-out evaluation on unseen towns.
- High recall on landable surfaces (0.92) with a model small enough for embedded hardware.
- Perception errors are measured and fed back into training (noise model), which made the learned landing controller robust to the real detector.
- Every mission-level claim is measured end to end with the real network in the loop, not only with offline metrics.

9.3 Limitations

- Sim-to-real gap: all images are synthetic; textures are procedural and simpler than real rubble. Real flight imagery will be needed before deployment.
- Close-range nadir accuracy (landable IoU ≈ 0.6 below 2 m) is limited by texture resolution and shadows.
- Low-contrast landable surfaces under some lighting are misclassified even at short range (observed in one procedural demo town).
- Mission success on harder towns is limited by geometry as much as by vision: if no landable surface exists at a street corner within radio range, the chain cannot turn that corner.
- Depth estimation (proposed in Review 1) was not used; the ground plane is assumed flat and obstacles come from range sensors.

9.4 Failure analysis of the live demonstration

During the live demonstration two maps failed, although they had succeeded in earlier headless tests. Each demonstration map was therefore re-evaluated over ten wind, noise and lighting seeds, and every failure was classified from logged map dumps as perception (a usable landable spot was observed but not recognised), exploration (a

usable spot was within radio range but never observed) or geometry (no usable spot existed). Nearly all failures were perception failures, and inspecting them revealed three vision-specific causes:

- **Camera pointing:** logged camera poses showed the drone approaching a corner while facing backwards (heading 223–275°); the body-fixed camera never saw the corner patch. Aiming the gimbal at the planner's goal fixed the worst map from 1/10 to 10/10.
- **Evidence fusion:** a patch seen correctly at 5 m was outvoted by dozens of earlier views from 20–25 m in which it was a blurred sliver; inverse-square range weighting lets one close look dominate.
- **Viewpoint coverage of training data:** white concrete in the shadow of three buildings, viewed obliquely, was under-represented; fine-tuning on patch-aimed views raised landable IoU on that view type from 0.787 to 0.874.

Table 13. Demonstration-map reliability over ten seeds each, before and after the vision fixes

Demonstration map	Before	After
Z-route	5/10	**10/10**
U-turn	9/10	**10/10**
Staircase	9/10	**10/10**
Blocked boulevard	5/10	**8/10**
Procedural town A (replaced, 5/10 → new town)	5/10	**10/10**
Procedural town B (level-3, replaced by a level-2 town)	1/10	**10/10**
Total	**34/60**	**58/60**

The two procedural towns were replaced after a robustness search: one original town contained a landable surface whose randomised colour made it indistinguishable from damaged street, and level-3 towns as a class are currently beyond the system (best 5/10). These are reported as limitations rather than hidden.

10. Conclusion

This project delivered a simulation-validated intelligent relay-drone system in which machine vision is the enabling component: drones find safe surfaces to perch on and thereby extend a disaster-area network one relay at a time. A fully synthetic, automatically labelled dataset and a 0.48 M-parameter U-Net achieved a test mIoU of 0.919 and a landable IoU of 0.825 on unseen towns. Geometric ground projection with geometry-weighted evidence fusion turned these predictions into a shared landable map,

and a connected-component locator provided centimetre-level landings under gusts when combined with a reinforcement-learning controller trained on the measured detector noise. With the real detector in the loop, the final system covered the camp in 45 % of 100 held-out missions (up from 34 %), extending coverage by 62 m on average.

The closed-loop experiments show quantitatively that perception quality determines communication range, and the failure analysis showed that where and how the camera looks, and how observations are fused, matter as much as raw segmentation accuracy. The same pipeline is ready to be retrained on real imagery and deployed on the embedded hardware proposed in Review 1.

11. Future Scope

- **Hardware deployment:** port the U-Net to the RV1103 NPU (INT8 quantisation) and integrate with the Pixhawk flight controller and gimbal proposed in Review 1.
- **Real data and sim-to-real transfer:** collect a small annotated set of real aerial images of damaged urban areas, fine-tune with mixed synthetic and real data, and add photorealistic rendering or image-translation-based domain adaptation.
- **Depth and geometry:** add monocular depth (e.g. a FastDepth-style encoder) or the ToF sensor to detect slopes and small obstacles that a flat-ground assumption misses.
- **Temporal and uncertainty-aware perception:** video segmentation and calibrated per-pixel uncertainty, so the fusion can trust confident pixels more and request a closer look when unsure.
- **Active perception:** let the planner choose viewpoints that maximise information about candidate spots instead of a fixed pan pattern.
- **Multi-drone extensions:** drones that re-position after landing, and coverage of several user points at the camp.

12. References

[1] FSO-Based Drone Assisted Mobile Access Network for Emergency Communications (surveyed in Review 1).

[2] Multi-UAV Wireless Mesh Networks for Disaster Recovery, 2026 (surveyed in Review 1).

[3] Blockchain-Enhanced UAV Networks for Post-Disaster Communication, 2026 (surveyed in Review 1).

[4] J. Long, E. Shelhamer and T. Darrell, "Fully Convolutional Networks for Semantic Segmentation," in Proc. IEEE CVPR, 2015.

[5] O. Ronneberger, P. Fischer and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in Proc. MICCAI, 2015.

[6] F. Milletari, N. Navab and S.-A. Ahmadi, "V-Net: Fully Convolutional Neural Networks for Volumetric Medical Image Segmentation," in Proc. 3DV, 2016.

[7] D. Wofk, F. Ma, T.-J. Yang, S. Karaman and V. Sze, "FastDepth: Fast Monocular Depth Estimation on Embedded Systems," in Proc. IEEE ICRA, 2019.

[8] S. R. Richter, V. Vineet, S. Roth and V. Koltun, "Playing for Data: Ground Truth from Computer Games," in Proc. ECCV, 2016.

[9] G. Ros, L. Sellart, J. Materzynska, D. Vazquez and A. M. Lopez, "The SYNTHIA Dataset: A Large Collection of Synthetic Images for Semantic Segmentation of Urban Scenes," in Proc. IEEE CVPR, 2016.

[10] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba and P. Abbeel, "Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World," in Proc. IEEE/RSJ IROS, 2017.

[11] T. Patterson, S. McClean, P. Morrow, G. Parr and C. Luo, "Timely Autonomous Identification of UAV Safe Landing Zones," Image and Vision Computing, vol. 32, no. 9, 2014.

[12] M. Mittal, R. Mohan, W. Burgard and A. Valada, "Vision-Based Autonomous UAV Navigation and Landing for Urban Search and Rescue," in Proc. ISRR, 2019.

[13] A. Elfes, "Using Occupancy Grids for Mobile Robot Perception and Navigation," Computer, vol. 22, no. 6, 1989.

[14] B. Yamauchi, "A Frontier-Based Approach for Autonomous Exploration," in Proc. IEEE CIRA, 1997.

[15] J. Schulman, F. Wolski, P. Dhariwal, A. Radford and O. Klimov, "Proximal Policy Optimization Algorithms," arXiv:1707.06347, 2017.

[16] A. Juliani et al., "Unity: A General Platform for Intelligent Agents," arXiv:1809.02627, 2018.

[17] T. S. Rappaport, Wireless Communications: Principles and Practice, 2nd ed., Prentice Hall, 2002.

[18] ITU-R Recommendation P.2040, "Effects of building materials and structures on radiowave propagation above about 100 MHz."